Paulina Bulanda 30.10.2026

# Rested / Restless – Project Documentation

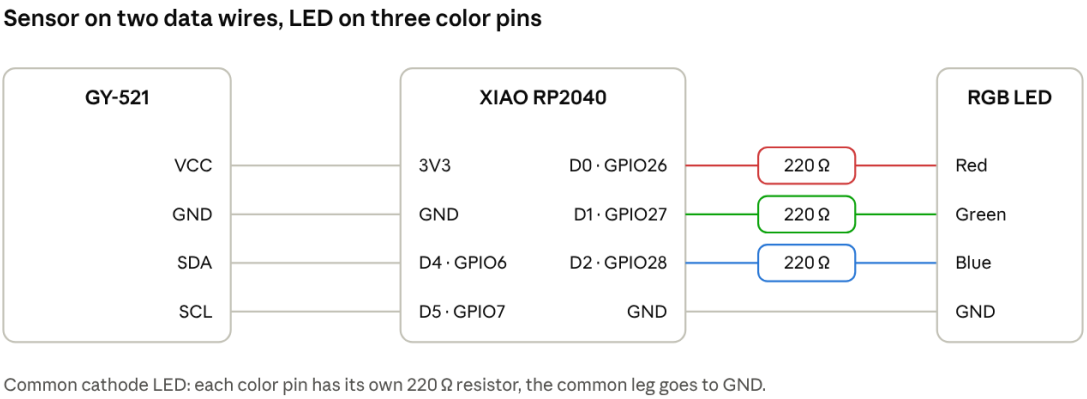
## Project idea

Rested / Restless turns the restlessness of a movement into colored light. A small motion sensor on a UR10e robot arm measures how the arm moves, an RGB LED shows the result (blue = calm, green = in between, red = wild), and a long-exposure photo records the light trail while the arm writes a word. The arm writes RESTED with a calm movement (blue trail) and RESTLESS with a restless movement (red and green trail). The question behind it: can we see how restless a movement is?

## Hardware and wiring

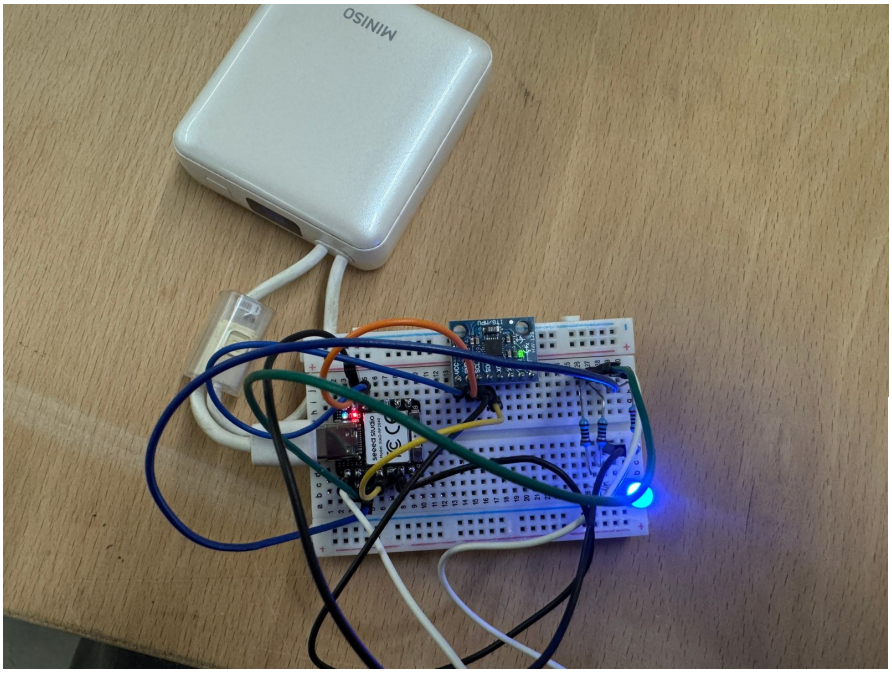
The electronics are one small board with a motion sensor and an RGB LED. I first powered it with a powerbank, so that nothing would be connected to a laptop while the robot moved. The powerbank, however, only kept the LED lit for a few seconds, so I switched back to powering the board from my laptop.

| Part | Job |
|---|---|
| Seeed XIAO RP2040 | Runs the MicroPython code |
| GY-521 (MPU6050) | Measures acceleration on x, y, z |
| RGB LED | Shows blue, green or red light |
| 3 × 220 Ω resistors | One in front of each LED color |
| UR10e robot arm | Moves the board and LED along the path |




Wiring diagram: Sensor and RGB LED on the XIAO

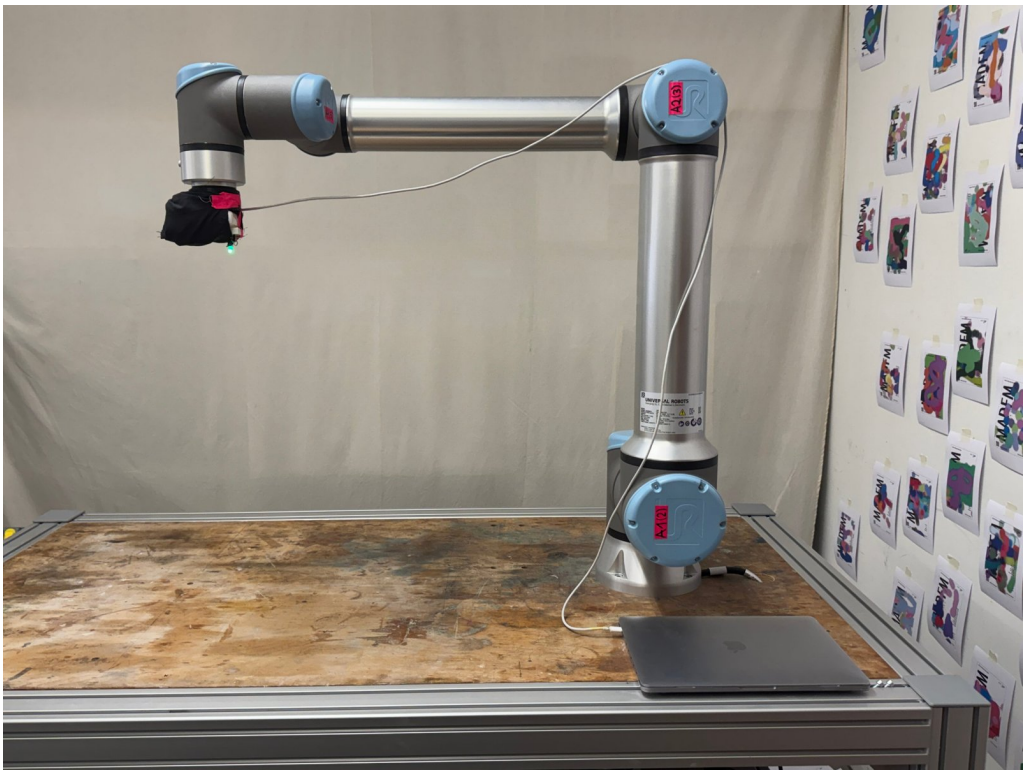
The sensor shares two data wires, and each LED color has its own pin and resistor.





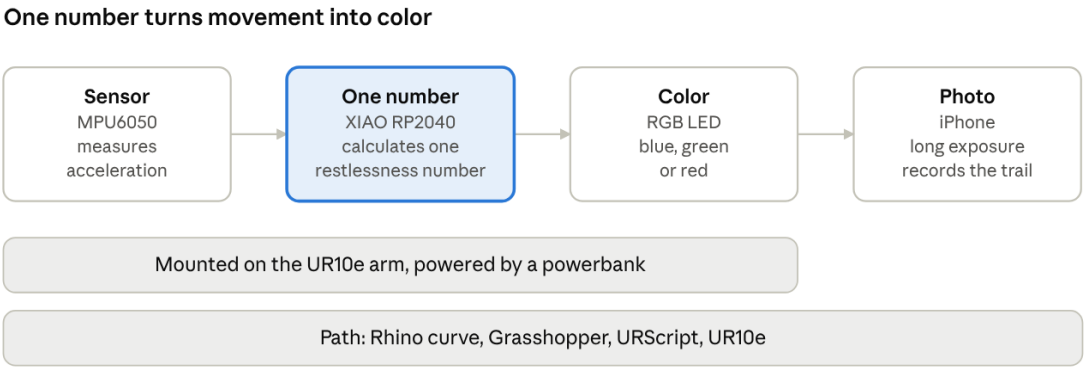


Left) Breadboard setup: XIAO RP2040, GY-521, RGB LED with resistors, and the powerbank
Right) The GY-521 (MPU6050) motion sensor module



The board with sensor and LED mounted at the end of the UR10e arm

## How it works




System chain, 4 steps

The sensor measures acceleration, the XIAO turns it into one "restlessness" number, the LED shows that number as a color, and the phone records the light trail while the arm moves.

## Software

The code is MicroPython running on the XIAO (written and uploaded with Thonny). It reads the sensor, turns the three acceleration values into one "restlessness" number and switches the LED color. The project was built in small steps, one script per step.

| File | What it does |
|---|---|
| `02_imu_read.py` | Reads x, y, z acceleration and prints them |
| `03_restlessness.py` | Combines x, y, z into one smoothed restlessness number |
| `04_read_serial.py` | Reads that number on the laptop with pySerial |
| `05_led_color.py` | Maps the number to blue, green or red on the LED |
| `main.py` | Final program on the XIAO, starts automatically with the powerbank |
| `library/imu.py`, `library/vector3d.py` | MPU6050 driver (third party, see Github sources) |

The accelerometer measures acceleration in g, not speed. Lying still it reads about 1 g (gravity), so restlessness is the distance from that value, smoothed so that the LED does not flicker:

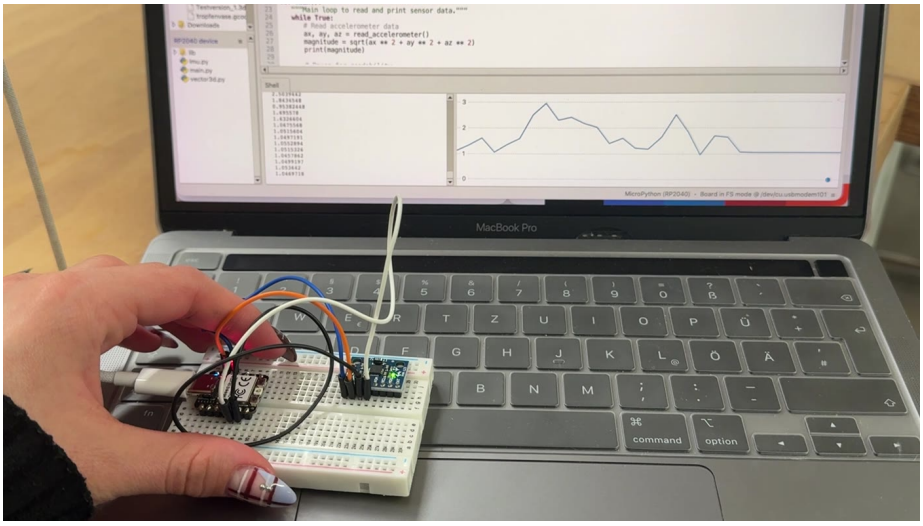
```
magnitude = sqrt(ax ** 2 + ay ** 2 + az ** 2)
restlessness = abs(magnitude - 1.055)      # 1.055 g = reading at rest
smoothed = SMOOTHING * smoothed + (1 - SMOOTHING) * restlessness

if smoothed >= RED_AT:
    color = (1, 0, 0)      # red: wild
elif smoothed >= GREEN_AT:
    color = (0, 1, 0)      # green: medium
else:
    color = (0, 0, 1)      # blue: calm

red.value(color[0])
green.value(color[1])
blue.value(color[2])
```

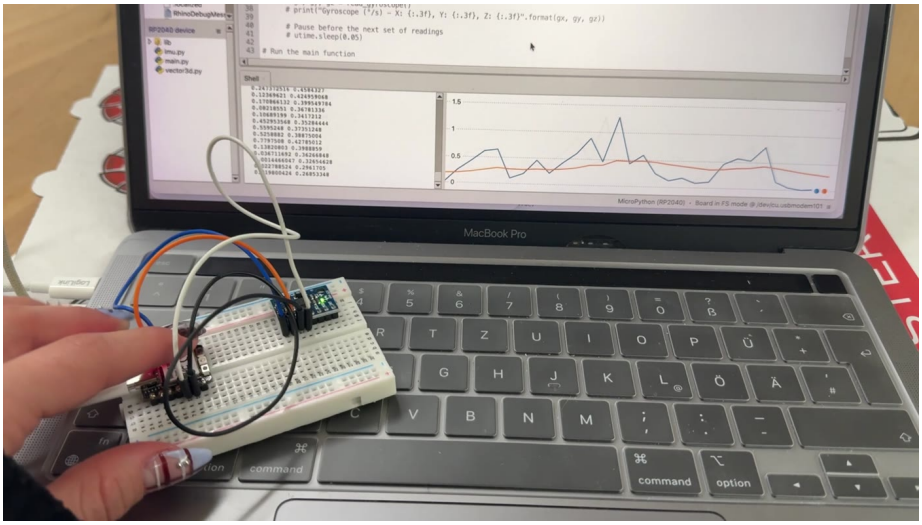
The values were tuned by testing on the robot:

| Setting | Value | Effect |
|---|---|---|
| `GREEN_AT` | 0.08 | From here on the LED is green |
| `RED_AT` | 0.17 | From here on the LED is red |
| `SMOOTHING` | 0.7 | Higher is smoother and slower |
| Loop pause | 0.05 s | About 20 readings per second |




Video file: `images/video_first_measuring.mp4`

First measurement with the sensor. The plotter shows the total acceleration, about 1 at rest, and it changes when the board is moved.




Video file: `images/video_zeroed_smoothed.mp4`

Measurement after setting the rest value to zero and smoothing the number. The orange line is the smoothed number.

## Robot path

The words are drawn as curves in Rhino and turned into a robot program in Grasshopper. The UR10e then moves the board along the path.

1. **Rhino:** draw the word as one open curve in the front view (XZ plane). RESTED and RESTLESS are separate curves.
2. **Grasshopper (`basic_ur_setup.gh`):** the curve goes into *Divide Curve*, which turns it into evenly spaced points. A higher count gives a smoother line (starts at 300 points).
3. Each point becomes a plane. A script component writes one URScript line per plane: `movel(p[...], a=ACC, v=VEL, r=BLEND)`.
4. The generated code is copied with *Copy Data Only* from the CODE panel and saved as a `.script` file.
5. The file is copied to a USB stick and loaded on the robot's teach pendant (*Load Program*), then started.

| Parameter | Meaning | RESTED | RESTLESS |
|---|---|---|---|
| `ACC` | Acceleration of the arm (m/s²) | 0.3 | 2 |
| `VEL` | Speed of the arm (m/s) | 0.15 | 0.5 |
| `BLEND` | Blend radius: how much corners are rounded (m) | 0.03 | 0 |
| Points on curve | Divide Curve count | 300 | 300 |

Two things mattered most. The blend radius must be smaller than the distance between two points, otherwise the robot does not run the program. And the writing must fit the arm's reach (about 1.3 m), so I had to set the letters closer together in Rhino.

## Photographing the light trail

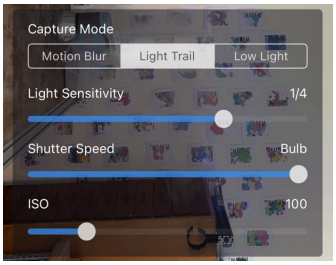
The light trail is captured with a long-exposure photo taken on an iPhone 15 Pro in a dark room. The phone stays fixed while the arm writes, so only the light moves in the picture.

- **App:** Slow Shutter, a long-exposure camera app, in Light Trail mode (settings below)
- **Light sensitivity:** 1/4
- **Shutter speed:** Bulb, so the exposure runs until I stop it
- **ISO:** 100, so the LED does not burn out to white
- **Focus:** locked on the robot plane before starting
- **Procedure:** start the exposure, start the program, stop when the arm has finished the word

The first tries were taken in a bright room, where the room light washes out the trail and the arm shows up in the picture. The final photos use the adjusted app settings in a completely dark room.



First tries with the robotic arm, taken in a bright room




The photo app with the settings used for the final photos

## Results

The same robot and the same technique produce two clearly different images: RESTED stays blue, RESTLESS switches between blue, green and red.




RESTED: final photo with the right settings in a completely dark room. The light stays blue.




RESTLESS: final photo, same settings and room. The color keeps changing, mostly between red and green.

Videos of the robotic arm writing RESTED and RESTLESS are available on GitHub.

Files: `images/video_rested.mp4` `images/video_restless.mp4`

## What I learned

- A library saves a lot of time. The MPU6050 driver handled the sensor, so I could focus on the restlessness logic
- An accelerometer measures acceleration, not speed, so the restlessness number has to be calculated from it
- Raw sensor values jump a lot. Smoothing the number kept the LED from flickering
- Testing in small steps (sensor, number, LED, robot) made it easy to find where something went wrong
- Code on a microcontroller runs on the board, not on the laptop. A `main.py` on the board starts by itself when the board gets power. A powerbank only lit the LED for a few seconds, so the board runs on the laptop's power instead
- A robot path has real limits: reach, point spacing and blend radius. The lettering had to be made narrower to fit

## Setup instructions

To rebuild the project you need the parts from the hardware table, a laptop with Thonny, and access to a UR10e with Rhino and Grasshopper.

1. **Wire the board.** Connect the sensor and LED as in the wiring diagram. Check the LED with `05_led_color.py` before mounting anything.
2. **Install MicroPython** on the XIAO RP2040 and open the board in Thonny.
3. **Copy the sensor library.** Upload `imu.py` and `vector3d.py` from the `library` folder to the board.
4. **Test the sensor.** Run `02_imu_read.py`. The values should be near 1 g in total when the board lies still.
5. **Test the number.** Run `03_restlessness.py` and shake the board. The value should be near 0 at rest and rise when you shake it.
6. **Install the final program.** Save `main.py` on the board. Unplug the USB cable and plug in the powerbank; the LED should turn on by itself.
7. **Mount the board and LED** on the robot arm. Connect to a power source.

8. **Create the robot program.** Draw the word in Rhino, open `basic_ur_setup.gh`, set the curve, point count, `ACC`, `VEL` and `BLEND`, copy the CODE panel and save it as a `.script` file.
9. **Run it.** Load the file from a USB stick on the teach pendant and start it while the phone takes the long-exposure photo.

## Code and project files

The Python code, diagrams, robot files, photos, videos and all sources are in the GitHub repository: https://github.com/paulinabul/rested-restless